

Step 4 — Plan expérimental

SmartReview : Machine Learning for Customer Review Analysis

1. Type de recherche retenu

Pour ce projet, nous avons opté pour une **recherche expérimentale quantitative**. Ce choix se justifie par la nature de notre problématique : nous cherchons à comparer les performances de quatre algorithmes de Machine Learning sur un même jeu de données, en utilisant des métriques objectives et mesurables. Ce type de recherche permet d'obtenir des résultats reproductibles et comparables aux travaux existants (*Puh & Bagic Babac, 2023 ; Mao et al., 2024*).

2. Activité A — Roadmap expérimental

Architecture du système SmartReview

1 — Données brutes	Dataset IMDb 50K (Kaggle) — avis annotés positif / négatif
2 — Prétraitement NLP	Nettoyage → Tokenisation → Stopwords → Lemmatisation
3 — Vectorisation	TF-IDF (scikit-learn, max_features=5000, ngram=(1,2))
4 — Modèles	Naive Bayes Régression Logistique SVM Random Forest
5 — Évaluation	Accuracy · Precision · Recall · F1-score · Temps d'exécution
6 — Application	Dashboard Streamlit — saisie d'avis → prédiction en temps réel

Jalons et étapes chronologiques

Jalon	Étape	Outils	Critère de validation
J1	Collecte et chargement du dataset	Kaggle, Python, pandas	Dataset chargé, classes équilibrées vérifiées
J2	Prétraitement NLP	NLTK, spaCy	Texte propre, sans bruit ni stopwords
J3	Vectorisation TF-IDF	scikit-learn	Matrice TF-IDF générée
J4	Entraînement des 4 modèles	scikit-learn	4 modèles entraînés sur même split
J5	Évaluation et comparaison	scikit-learn, matplotlib	Tableau comparatif des métriques produit
J6	Comparaison aux benchmarks	Puh & Babac (2023)	Résultats situés par rapport à la littérature
J7	Développement Streamlit	Streamlit	Application fonctionnelle testée
J8	Rédaction et validation	Word	Rapport final complet et relu

3. Activité B — Variables et métriques

Dans cette expérimentation, nous distinguons deux types de variables. Les **variables indépendantes** sont celles que nous contrôlons et faisons varier, tandis que les **variables dépendantes** sont celles que nous observons et mesurons pour constater l'effet produit.

Type	Variable	Définition	Valeurs / Indicateurs
Indépendante	Algorithme ML utilisé	L'algorithme que l'on teste	NB / LR / SVM / RF
Indépendante	Prétraitement NLP	Présence ou absence du prétraitement	Avec / Sans
Indépendante	Représentation textuelle	Méthode de vectorisation choisie	TF-IDF
Dépendante	Accuracy	% global de prédictions correctes	0 – 100 %
Dépendante	Precision	Vrais positifs / total positifs prédits	0 – 1

Dépendante	Recall	Vrais positifs / total positifs réels	0 – 1
Dépendante	F1-score	Moyenne harmonique Precision/Recall	0 – 1
Dépendante	Temps d'exécution	Durée d'entraînement + prédiction	Secondes

Le choix du **F1-score comme métrique principale** se justifie par le fait que notre dataset peut présenter un léger déséquilibre entre avis positifs et négatifs. Dans ce cas, l'accuracy seule peut être trompeuse. Le temps d'exécution a été ajouté car il représente un critère important pour les entreprises disposant de ressources limitées.

4. Activité C — Protocole expérimental

Instruments et outils

- **Dataset** : IMDb 50K Movie Reviews (Kaggle) — 25 000 avis positifs / 25 000 négatifs, annoté, équilibré et public.
- **Langage** : Python 3.x sous Google Colab / Jupyter Notebook.
- **Librairies** : scikit-learn (modèles + TF-IDF), NLTK / spaCy (prétraitement), pandas (données), matplotlib / seaborn (visualisation), Streamlit (application).
- **Versioning** : Code disponible sur GitHub pour garantir la reproductibilité.

Protocole étape par étape

Étape 1 — Collecte : Charger le dataset IMDb depuis Kaggle, vérifier l'équilibre des classes et explorer les données (longueur des avis, distribution des labels).

Étape 2 — Prétraitement : Conversion en minuscules → suppression ponctuation et caractères spéciaux → tokenisation → suppression stopwords → lemmatisation (*Jurafsky & Martin, 2023*).

Étape 3 — Vectorisation : Application de TF-IDF (max_features=5000, ngram_range=(1,2)) via TfidfVectorizer de scikit-learn (*Puh & Bagic Babac, 2023*).

Étape 4 — Split : Division du dataset : 80 % entraînement / 20 % test (random_state=42 pour assurer la reproductibilité).

Étape 5 — Entraînement : Entraînement des 4 modèles (NB, LR, SVM, RF) sur les mêmes données d'entraînement avec les paramètres par défaut documentés.

Étape 6 — Évaluation : Calcul de Accuracy, Precision, Recall, F1-score et temps d'exécution pour chaque modèle sur les données de test.

Étape 7 — Comparaison : Confrontation des résultats obtenus aux benchmarks de la littérature : SVM 80 %, NB 73 % (*Puh & Bagic Babac, 2023*).

Preuve de concept et validation des hypothèses

Hypothèse	Comment la valider ?	Critère de confirmation
H1 — SVM meilleur	Comparer F1-score des 4 modèles	F1 (SVM) > F1 (NB), F1 (LR), F1 (RF)
H2 — Prétraitement améliore	Entraîner avec et sans prétraitement	F1 avec prétraitement > F1 sans prétraitement
H3 — LR meilleur compromis	Comparer F1 + temps d'exécution	LR : F1 élevé ET temps faible vs autres

5. Analyse de validité — Biais et limites

Biais identifié	Description	Comment le minimiser ?
Biais de domaine	Dataset IMDb limité au domaine cinéma — résultats pas généralisables à tous les avis	Tester aussi sur Amazon Reviews (multi-domaines)

Biais de paramètres	Utilisation des hyperparamètres par défaut peut défavoriser certains modèles	Documenter tous les paramètres, envisager grid search
Biais de split	Le découpage train/test peut influencer les résultats	Fixer random_state=42, envisager cross-validation
Biais de représentation	TF-IDF ne capture pas le contexte sémantique (ex : ironie, négation)	Mentionner comme limite, comparer à GloVe en perspective
Limite de généralisation	Résultats valides sur IMDb 50K uniquement	Déclarer explicitement le périmètre de l'étude

Malgré ces limites, notre protocole garantit la **reproductibilité** grâce au dataset public, au random_state fixe et au code versionné sur GitHub. Un autre chercheur disposant des mêmes outils pourrait reproduire exactement les mêmes résultats.